Product of Array Except Self

1. Problem Title & Link

Title: **LeetCode 238 : Product of Array Except Self**

Link: https://leetcode.com/problems/product-of-array-except-self/

2. Problem Statement (Clear Explanation)

You are given an integer array nums of length n.

Your task is to return an array answer such that:

answer[i] = product of all elements of nums except nums[i]

In other words, for each index i, compute the **product of every number in the array except the one at index i**.

Important conditions:

1. **You cannot use division**.
2. The solution must run in **O(n) time**.
3. The algorithm should use **constant extra space** (excluding the output array).

The output array must contain the product values for each index.

3. Examples (Input → Output)

Example 1

Input

nums = [1,2,3,4]

Output

[24,12,8,6]

Explanation

Index	Calculation	Result
0	2×3×4	24
1	1×3×4	12
2	1×2×4	8
3	1×2×3	6

Example 2

Input

nums = [-1,1,0,-3,3]

Output

[0,0,9,0,0]

Explanation

Because one element is **0**, the product for all other positions becomes **0**, except the position of the zero itself.

4. Constraints

2 ≤ nums.length ≤ 100000

-30 ≤ nums[i] ≤ 30

Additional conditions:

- Product of any prefix or suffix fits in a **32-bit integer**
- Division operation **is not allowed**

5. Core Concept (Pattern / Topic)

Prefix Product

Suffix Product

Array Traversal

6. Thought Process (Step-by-Step Explanation)

Step 1 — Brute Force Idea

For every index i, multiply all other elements.

Example:

nums = [1,2,3,4]

For i = 0

2×3×4

For i = 1

1×3×4

Time complexity:

$O(n^2)$

Too slow.

Step 2 — Key Observation

Instead of recomputing the product each time, we can split the calculation into:

prefix product

suffix product

For index i:

answer[i] = product of elements before i × product of elements after i

Step 3 — Prefix Product

Example:

nums = [1,2,3,4]

Prefix products:

[1,1,2,6]

Explanation:

prefix[i] = nums[0] * nums[1] * ... * nums[i-1]

Step 4 — Suffix Product

Suffix products:

[24,12,4,1]

Explanation:

suffix[i] = nums[i+1] * nums[i+2] * ... * nums[n-1]

Step 5 — Final Result

answer[i] = prefix[i] × suffix[i]

Example:

prefix = [1,1,2,6]

suffix = [24,12,4,1]

result = [24,12,8,6]

7. Visual / Intuition Diagram

Example

nums = [1,2,3,4]

Index 0 1 2 3

Nums 1 2 3 4

Prefix 1 1 2 6

Suffix 24 12 4 1

Result 24 12 8 6

8. Pseudocode (Language Independent)

```
function productExceptSelf(nums):

    n = length(nums)

    answer = array of size n

    prefix = 1
```

```
    for i from 0 to n-1:
        answer[i] = prefix
        prefix *= nums[i]

    suffix = 1

    for i from n-1 to 0:
        answer[i] *= suffix
        suffix *= nums[i]

    return answer
```

9. Code Implementation

Python

```
def productExceptSelf(nums):

    n = len(nums)
    answer = [1] * n

    prefix = 1

    for i in range(n):
        answer[i] = prefix
        prefix *= nums[i]

    suffix = 1

    for i in range(n-1, -1, -1):
        answer[i] *= suffix
        suffix *= nums[i]

    return answer
```

Java

```
class Solution {

    public int[] productExceptSelf(int[] nums) {
```

```
        int n = nums.length;

        int[] answer = new int[n];

        answer[0] = 1;

        for(int i = 1; i < n; i++) {
            answer[i] = answer[i-1] * nums[i-1];
        }

        int suffix = 1;

        for(int i = n-1; i >= 0; i--) {

            answer[i] *= suffix;

            suffix *= nums[i];
        }

        return answer;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

Two linear passes through the array.

Space Complexity

O(1)

(Extra space not counting output array)

11. Common Mistakes / Edge Cases

Common mistakes:

1. Using **division**
2. Not handling **zero values**

Example:

nums = [1,2,0,4]

3. Incorrect prefix initialization.

Correct initialization:

prefix = 1

suffix = 1

12. Detailed Dry Run

Example

nums = [1,2,3,4]

Prefix pass:

i	prefix	answer
0	1	[1,,,_]
1	1	[1,1,,]
2	2	[1,1,2,_]
3	6	[1,1,2,6]

Suffix pass:

i	suffix	answer
3	1	[1,1,2,6]
2	4	[1,1,8,6]
1	12	[1,12,8,6]
0	24	[24,12,8,6]

13. Common Use Cases (Real-Life / Interview)

This pattern appears in:

- **cumulative calculations**
- **prefix/suffix analytics**
- **financial portfolio analysis**
- **parallel computation optimizations**

14. Common Traps (Important!)

1. Using division
2. Incorrect prefix initialization
3. Forgetting suffix pass

15. Builds To (Related LeetCode Problems)

Progression:

LC 1480 Running Sum of Array

LC 238 Product of Array Except Self

LC 560 Subarray Sum Equals K

LC 523 Continuous Subarray Sum

LC 724 Find Pivot Index

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	O(1)	Too slow
Division	O(n)	O(1)	Not allowed
Prefix + Suffix	O(n)	O(1)	Optimal

17. Why This Solution Works (Short Intuition)

For each index, we multiply the **product of all numbers before it** and the **product of all numbers after it**, which together represent the product of all elements except the current one.

18. Variations / Follow-Up Questions

Possible interview extensions:

1. Handle **multiple zeros**
2. Return **prefix products**
3. Solve using **parallel processing**
4. Extend to **2D matrix products**.
5.